Java Process Table Function (PTF) User-Defined Function (UDF) type — Timer-driven PTF examples

> This package contains **three timer-driven PTF UDFs** bundled into a single JAR:
>
> 1. **Session Timeout Detector** — uses **named timers** to implement the inactivity pattern (only the latest timer survives)
> 2. **Per-Event Follow-Up** — uses **unnamed timers** to schedule independent follow-ups for every event (all timers fire)
> 3. **SLA Monitor** — uses **unnamed timers** to enforce per-request SLA deadlines and detect breaches
>
> Unlike the companion User Event Enricher (which is purely row-driven — state transitions triggered only by incoming rows), these PTFs use Flink's timer service to schedule future actions and react when those timers fire.
>
> This example demonstrates how the Process Table Function (PTF) API in Flink 2.1+ enables building fully stateful, **timer-driven** operators in Java that are directly callable from SQL.

> ***Note: The `ArgumentTrait.REQUIRE_ON_TIME` trait is not yet supported in Confluent Cloud early access. Please see the Known Issues section for details.***

Table of Contents

- **1.0 Timers and timer-driven processing in a Process Table Function**
  - **1.1 What are timers?**
  - **1.2 How timers work in a PTF**
  - **1.3 Named timers and the inactivity pattern**
  - **1.4 Unnamed timers and the scheduling pattern**
  - **1.5 The role of `on_time`, `TimeContext`, and `OnTimerContext`**
- **2.0 UDF 1: Session Timeout Detector (named timers)**
  - **2.1 Detection logic**
  - **2.2 How it works end-to-end**
  - **2.3 Key concepts illustrated**
- **3.0 UDF 2: Per-Event Follow-Up (unnamed timers)**
  - **3.1 Follow-up logic**
  - **3.2 How it works end-to-end**
  - **3.3 Key concepts illustrated**
- **4.0 UDF 3: SLA Monitor (unnamed timers)**
  - **4.1 Monitoring logic**
  - **4.2 How it works end-to-end**
  - **4.3 Key concepts illustrated**
- **5.0 Comparing the UDFs in this package**
- **6.0 Row-driven vs timer-driven — comparing with the row-driven PTF example**
- **7.0 Resources**

1.0 Timers and timer-driven processing in a Process Table Function

A **Process Table Function (PTF)** is a stateful operator that sits inside the Flink dataflow graph. The companion example showed how a PTF can maintain per-user state and transition that state purely in response to incoming rows. This example adds a second dimension: **time**.

1.1 What are timers?

In stream processing, **timers** let an operator schedule a callback at a future point in time. Without timers, an operator can only react when a new row arrives. With timers, it can also react when *nothing happens* — detecting inactivity, enforcing deadlines, or triggering periodic side-effects.

Flink manages timers as a special kind of state:

- Timers are persisted in the **state backend** alongside your other state
- They survive **checkpoints and restores** — if the job restarts, pending timers are restored and fire at the correct time
- They are scoped to a **partition key** — each `PARTITION BY` key has its own independent set of timers

1.2 How timers work in a PTF

In a PTF, timers are accessed through the `TimeContext` obtained from the `Context` parameter:

```
TimeContext<Instant> timeCtx = ctx.timeContext(Instant.class);

// Register a named timer that fires 5 minutes from now
timeCtx.registerOnTime("inactivity",
timeCtx.time().plus(Duration.ofMinutes(5)));
```

When the timer fires, Flink calls the `onTimer()` method on the same PTF instance with the same partition key's state:

```
public void onTimer(OnTimerContext onTimerCtx, SessionState state) {
    // Timer fired — the user has been inactive for 5 minutes
    collect(Row.of("session_timeout", state.lastPayload, state.eventCount,
true));
}
```

1.3 Named timers and the inactivity pattern

Inactivity patterns *are design strategies for detecting and responding to the absence of expected events within a specific time frame*. In other words, "instead of reacting when something occurs, you react when it doesn't." For example:

- *Session timeout*: flag a user session as idle after no clicks or actions for 5 minutes (this example)
- *Abandoned cart*: trigger a reminder when a shopping cart has no activity for 24 hours
- *Device heartbeat monitoring*: alert when a sensor or server stops sending pings

Timers can be **named** or **unnamed**:

Type	Registration	Behaviour on re-register
Named	`registerOnTime("name", time)`	**Replaces** the existing timer with the same name
Unnamed	`registerOnTime(time)`	**Adds** a new timer (previous ones still fire)

Named timers are crucial for the **inactivity pattern**: on each event, re-register a timer with the same name. Every new event overrides the previous timer, effectively resetting the inactivity period. If no new event occurs, the timer triggers.

1.4 Unnamed timers and the scheduling pattern

Scheduling patterns *are design strategies for triggering a future action in response to each incoming event*. In other words, "every event independently schedules its own deferred callback." For example:

- *Per-event follow-up*: schedule a reminder or notification for each user action (this example)
- *SLA monitoring*: each request gets its own SLA deadline timer — when it fires, check if the request was completed
- *Delayed side-effects*: trigger a downstream action (email, webhook, audit log entry) a fixed time after each event

Unnamed timers implement the scheduling pattern. Each call to `registerOnTime(time)` (without a name) **adds** a new independent timer — previous timers are not replaced:

```
TimeContext<Instant> timeCtx = ctx.timeContext(Instant.class);

// Each call adds a new timer — no replacement
timeCtx.registerOnTime(timeCtx.time().plus(Duration.ofMinutes(2)));
```

If three events arrive within the follow-up window, all three timers fire independently:

```
Event 1 at T+0s   → timer scheduled for T+2m     → fires at T+2m
Event 2 at T+30s  → timer scheduled for T+2m30s  → fires at T+2m30s
Event 3 at T+1m   → timer scheduled for T+3m     → fires at T+3m
```

1.5 The role of `on_time`, `TimeContext`, and `OnTimerContext`

Three pieces connect timers to the PTF:

- **`on_time => DESCRIPTOR(event_time)`** — the SQL-side declaration that tells Flink which column carries the event timestamp. This is required for the PTF to access `TimeContext`.
- **`@ArgumentHint(ArgumentTrait.REQUIRE_ON_TIME)`** — the Java-side annotation that declares the PTF requires an `on_time` argument. Without this, the `TimeContext` is not available.
- **`TimeContext<Instant>`** — provides `time()` (current event timestamp), `registerOnTime()`, `clearTimer()`, and `currentWatermark()`.

- **`OnTimerContext`** — passed to `onTimer()`, identifies which timer fired via `currentTimer()`.

2.0 UDF 1: Session Timeout Detector (named timers)

The `SessionTimeoutDetector` PTF reads user activity events from a Kafka topic (`user_activity`), monitors per-user inactivity using **named timers**, and writes enriched output (including timeout alerts) to a second Kafka topic (`timeout_events`).

2.1 Detection logic

For every incoming event the function maintains three pieces of **per-user state** (one state instance per `PARTITION BY user_id` key):

State field	Purpose
`eventCount`	Running count of events received in the current session. Reset to zero when the timer fires (session timeout).
`lastEventType`	The `event_type` of the most recent event. Used in the timeout row for context.
`lastPayload`	The `payload` of the most recent event. Carried forward in the timeout row.

The output schema is:

```
user_id      STRING    — passed through automatically via PARTITION BY
event_type   STRING    — original type, or "session_timeout" when timer
fires
payload      STRING    — original payload, or last-seen payload on timeout
event_count  BIGINT    — total events seen before this output
timed_out    BOOLEAN   — false for regular events, true for timeout events
```

2.2 How it works end-to-end

```
Kafka (user_activity)
        │
        ▼
  ┌──────────────┐
  │ user_activity │   Flink SQL source table (JSON / event-time watermark)
  └──────────────┘
        │
        ▼
  SessionTimeoutDetector(
      input   => TABLE user_activity PARTITION BY user_id,
      on_time => DESCRIPTOR(event_time)
  )
        │
        │  Per-user timer-driven detection:
        │    • every event → update state, reset 5-min timer, emit
enriched row
```

```
            │     • timer fires → emit "session_timeout" row, clear state
            │
            ▼
    ┌─────────────────┐
    │ timeout_events  │    Flink SQL sink table → Kafka (timeout_events)
    └─────────────────┘
```

Example timeline for user `alice`:

Time	Event	What happens	Output
T+0s	`login`	State: count=1. Timer set for T+5m.	`(login, web, 1, false)`
T+30s	`click`	State: count=2. Timer reset to T+5m30s.	`(click, button-home, 2, false)`
T+2m	`purchase`	State: count=3. Timer reset to T+7m.	`(purchase, order-42, 3, false)`
T+7m	*(timer fires)*	No event since T+2m.	`(session_timeout, order-42, 3, true)`

2.3 Key concepts illustrated

- **`ProcessTableFunction`** — the Flink 2.x API for stateful, set-semantic UDFs callable from SQL.
- **`@StateHint`** — declares a POJO whose fields Flink automatically persists per partition key.
- **`@ArgumentHint(ArgumentTrait.SET_SEMANTIC_TABLE, ArgumentTrait.REQUIRE_ON_TIME)`** — declares that the input is a keyed, stateful virtual processor that requires timer support via an `on_time` argument.
- **`TimeContext`** — provides access to the current event timestamp and timer registration/cancellation.
- **`onTimer()`** — the callback that fires when a registered timer elapses.
- **Named timers** — re-registering a timer with the same name replaces it, enabling the inactivity pattern.

3.0 UDF 2: Per-Event Follow-Up (unnamed timers)

The `PerEventFollowUp` PTF reads user action events from a Kafka topic (`user_actions`), schedules an independent follow-up for each event using **unnamed timers**, and writes enriched output (including follow-up events) to a second Kafka topic (`follow_up_events`).

3.1 Follow-up logic

For every incoming event the function maintains four pieces of **per-user state** (one state instance per `PARTITION BY user_id` key):

State field	Purpose
`eventCount`	Running count of events received. Never reset — grows monotonically.
`followUpCount`	Running count of follow-up events emitted. Grows as timers fire.

State field	Purpose
`lastEventType`	The `event_type` of the most recent event. Carried forward in follow-up rows.
`lastPayload`	The `payload` of the most recent event. Carried forward in follow-up rows.

The output schema is:

```
user_id          STRING    — passed through automatically via PARTITION BY
event_type       STRING    — original type, or "follow_up" when timer
fires
payload          STRING    — original payload, or last-seen payload on
follow-up
event_count      BIGINT    — total events seen before this output
follow_up_count  BIGINT    — total follow-ups emitted before this output
is_follow_up     BOOLEAN   — false for regular events, true for follow-up
events
```

3.2 How it works end-to-end

```
Kafka (user_actions)
        │
        ▼
  ┌──────────────┐
  │ user_actions │   Flink SQL source table (JSON / event-time watermark)
  └──────────────┘
        │
        ▼
  PerEventFollowUp(
      input   => TABLE user_actions PARTITION BY user_id,
      on_time => DESCRIPTOR(event_time)
  )
        │
        │  Per-user unnamed-timer processing:
        │    • every event → update state, register unnamed timer, emit
enriched row
        │    • each timer fires independently → emit "follow_up" row
        │
        ▼
  ┌─────────────────┐
  │ follow_up_events │   Flink SQL sink table → Kafka (follow_up_events)
  └─────────────────┘
```

Example timeline for user `alice`:

Time	Event	What happens	Output
T+0s	`login`	State: count=1. Unnamed timer set for T+2m.	`(login, web, 1, 0, false)`

Time	Event	What happens	Output
T+30s	`click`	State: count=2. New unnamed timer set for T+2m30s.	`(click, button-home, 2, 0, false)`
T+1m	`purchase`	State: count=3. New unnamed timer set for T+3m.	`(purchase, order-42, 3, 0, false)`
T+2m	*(timer 1 fires)*	Follow-up for login event.	`(follow_up, order-42, 3, 1, true)`
T+2m30s	*(timer 2 fires)*	Follow-up for click event.	`(follow_up, order-42, 3, 2, true)`
T+3m	*(timer 3 fires)*	Follow-up for purchase event.	`(follow_up, order-42, 3, 3, true)`

Notice that **all three timers fire** — unlike the Session Timeout Detector where only the last timer would have fired.

3.3 Key concepts illustrated

- **Unnamed timers** — each `registerOnTime(time)` call (without a name) adds a new independent timer. No replacement, no deduplication by name.
- **Additive state** — unlike the Session Timeout Detector which clears state on timeout, the Per-Event Follow-Up preserves state across follow-ups.
- **Multiple timer fires per key** — demonstrates that unnamed timers accumulate, producing one `onTimer()` callback per registered timer.

4.0 UDF 3: SLA Monitor (unnamed timers)

The `SlaMonitor` PTF reads service request events from a Kafka topic (`service_requests`), enforces per-request SLA deadlines using **unnamed timers**, and writes enriched output (including breach alerts) to a second Kafka topic (`sla_events`).

4.1 Monitoring logic

For every incoming request the function maintains four pieces of **per-request state** (one state instance per `PARTITION BY request_id` key):

State field	Purpose
`updateCount`	Running count of status updates received for this request.
`resolved`	Whether the request has been resolved. Set to `true` on `"resolved"` status.
`serviceName`	The service that owns this request. Carried forward in breach rows.
`lastStatus`	The most recent status of the request.

The output schema is:

```
request_id       STRING    — passed through automatically via PARTITION BY
status           STRING    — original status, or "sla_breach" when timer
fires
service_name     STRING    — the service that owns this request
update_count     BIGINT    — total status updates received
is_resolved      BOOLEAN   — whether the request has been resolved
is_breach        BOOLEAN   — false for regular events, true for SLA breach
events
```

4.2 How it works end-to-end

```
Kafka (service_requests)
        │
        ▼
  ┌──────────────────────┐
  │ service_requests     │   Flink SQL source table (JSON / event-time
watermark)
  └──────────────────────┘
        │
        │
        ▼
  SlaMonitor(
      input   => TABLE service_requests PARTITION BY request_id,
      on_time => DESCRIPTOR(event_time)
  )
        │
        │  Per-request unnamed-timer processing:
        │    • "opened" event → update state, register SLA deadline
timer, emit row
        │    • "resolved" event → mark resolved in state, emit row
        │    • timer fires → if not resolved, emit "sla_breach" row
        │
        ▼
  ┌────────────────┐
  │  sla_events    │   Flink SQL sink table → Kafka (sla_events)
  └────────────────┘
```

Example timeline for request `REQ-001`:

Time	Event	What happens	Output
T+0s	`opened`	State: count=1, resolved=false. Timer set for T+10m.	`(opened, billing, 1, false, false)`
T+2m	`in_progress`	State: count=2, resolved=false.	`(in_progress, billing, 2, false, false)`
T+10m	*(timer fires)*	Not resolved → emit SLA breach.	`(sla_breach, billing, 2, false, true)`

Example timeline for request `REQ-002` (resolved in time):

Time	Event	What happens	Output
T+0s	`opened`	State: count=1, resolved=false. Timer set for T+10m.	`(opened, payments, 1, false, false)`
T+5m	`resolved`	State: count=2, resolved=true.	`(resolved, payments, 2, true, false)`
T+10m	*(timer fires)*	Already resolved → **no output**.	*(nothing emitted)*

4.3 Key concepts illustrated

- **Scheduling pattern for SLA enforcement** — each request independently schedules its own deadline timer. The timer fires regardless of state changes; `onTimer()` checks whether the deadline was met.
- **Conditional output on timer fire** — unlike the other two UDFs that always emit on timer fire, the SLA Monitor only emits when the request is still unresolved. This demonstrates that `onTimer()` can inspect state and decide whether to produce output.
- **Unnamed timers with real-world semantics** — each `"opened"` request gets its own timer. Multiple requests under the same key (if partitioned differently) would each track independently.

5.0 Comparing the UDFs in this package

Aspect	Session Timeout Detector (named timers)	Per-Event Follow-Up (unnamed timers)	SLA Monitor (unnamed timers)
Timer type	Named (`"inactivity"`)	Unnamed	Unnamed
Re-register behaviour	**Replaces** the previous timer	**Adds** a new independent timer	**Adds** a new independent timer
Timers pending per key	At most 1	One per event (can accumulate)	One per `"opened"` request
Output per timer fire	1 timeout row (then clears state)	1 follow-up row (state preserved)	1 breach row **only if not resolved**
State on timer fire	Cleared (session ended)	Preserved (follow-ups are additive)	Preserved (checked for resolution)
Primary pattern	Inactivity / absence detection	Per-event delayed actions	SLA deadline enforcement
Use cases	Session timeout, abandoned cart, heartbeat	Reminders, delayed side-effects	SLA monitoring, deadline compliance

---

6.0 Row-driven vs timer-driven — comparing with the row-driven PTF example

Aspect	User Event Enricher (row-driven)	Timer-driven UDFs (this package)
Trigger	Every incoming row	Every incoming row **+ timer fire**
Uses timers	No	Yes (`TimeContext`, `onTimer()`)
`on_time` argument	Not required	**Required** (`DESCRIPTOR(event_time)`)
`@ArgumentHint` traits	`SET_SEMANTIC_TABLE`	`SET_SEMANTIC_TABLE` + `REQUIRE_ON_TIME`
State transitions	Row-driven only (login → new session)	Row-driven (update count, reset/add timer) + timer-driven (timeout/follow-up/breach)
Output trigger	One output per input row	One output per input row **+ zero or one output per timer fire**
Use case	Enrichment, session tracking	Inactivity detection, per-event follow-ups, SLA monitoring

7.0 Resources

- Apache Flink User-defined Functions
- Create a User-Defined Function with Confluent Cloud for Apache Flink
- Process Table Functions in Confluent Cloud for Apache Flink